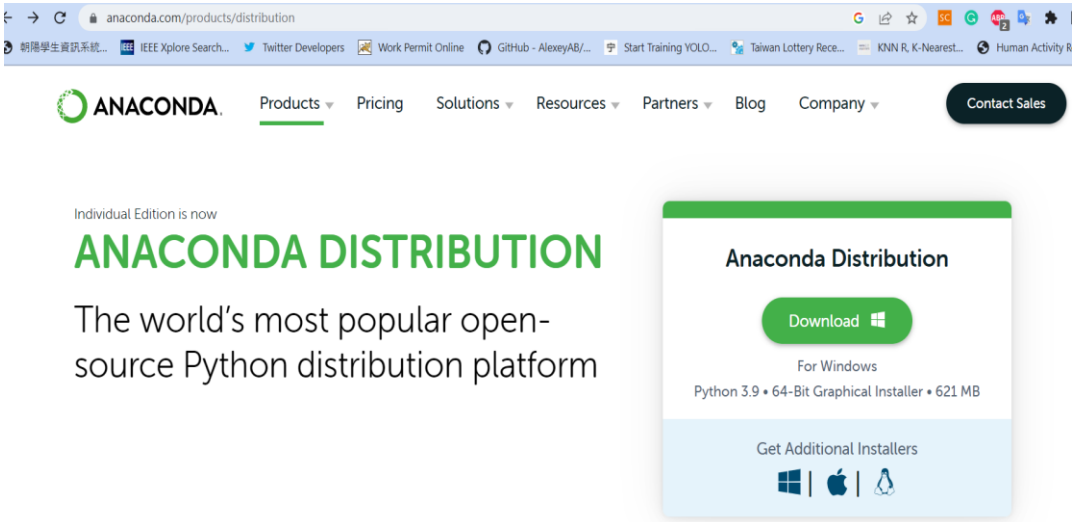
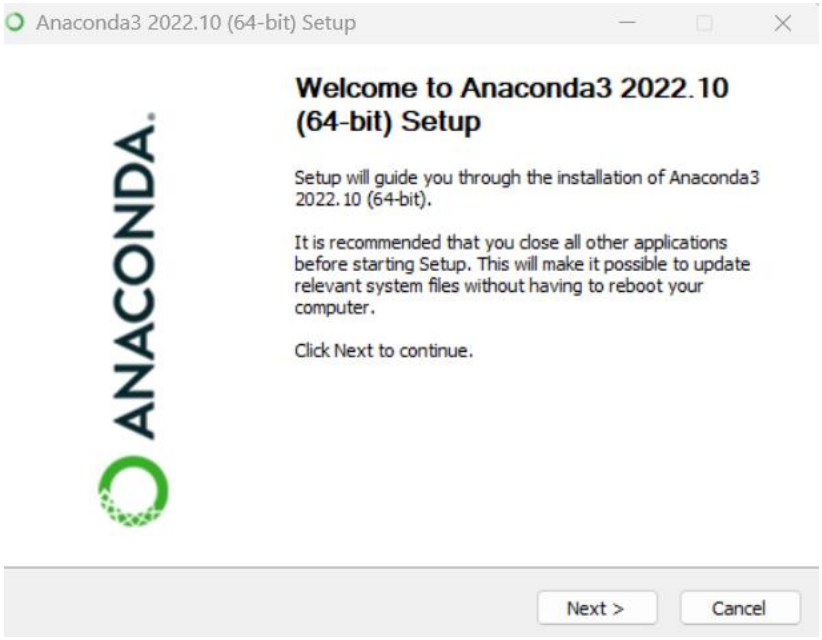
OBJECT RECOGNITION

By. Christine Dewi, Ph.D.
Department of Information Technology,
Satya Wacana Christian University Salatiga, Indonesia

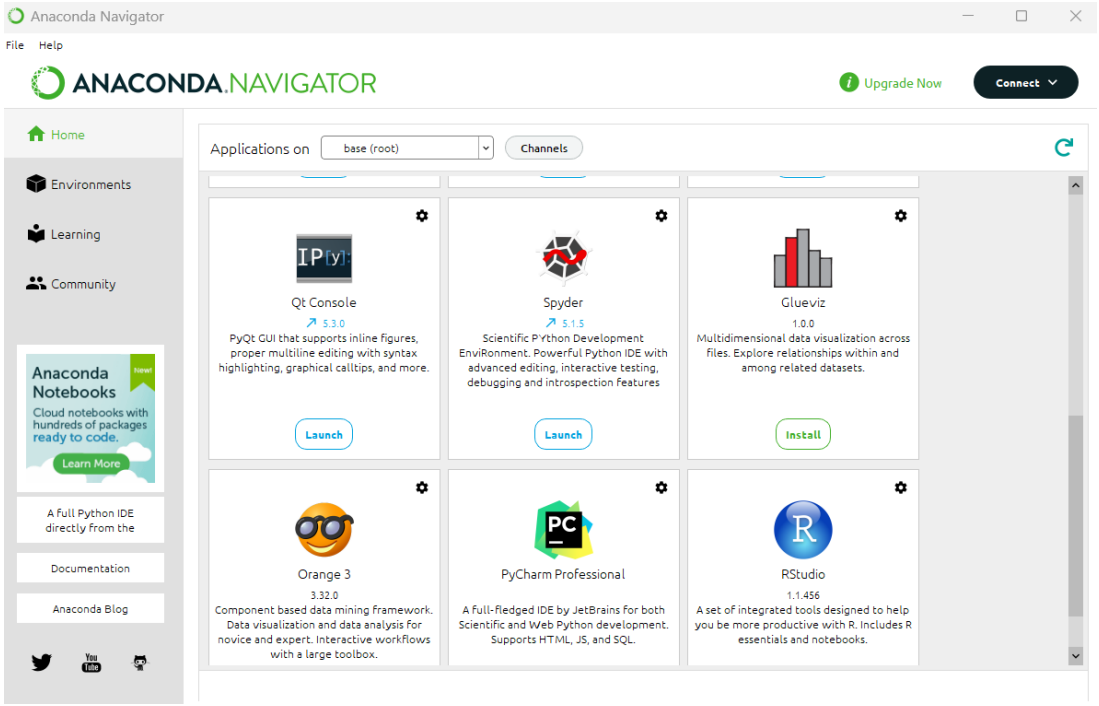
1. **Installs**
   a. Anaconda
   https://www.anaconda.com/products/distribution



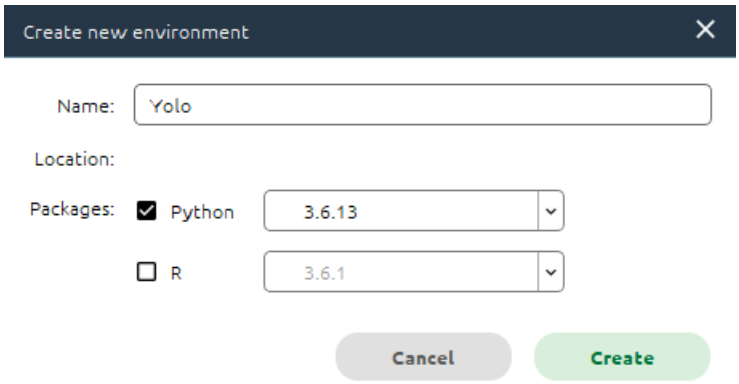
   b. Installation Process



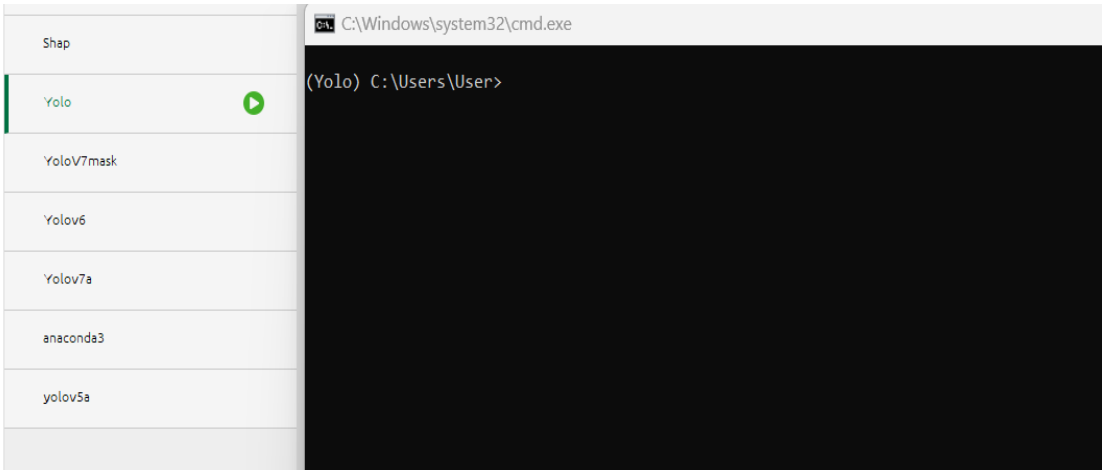
c. Installation finish, open your anaconda



d. Create new environment. Click tab environment >> create new environment >> name: Yolo >> Packages Python version 3.6.13 >> create



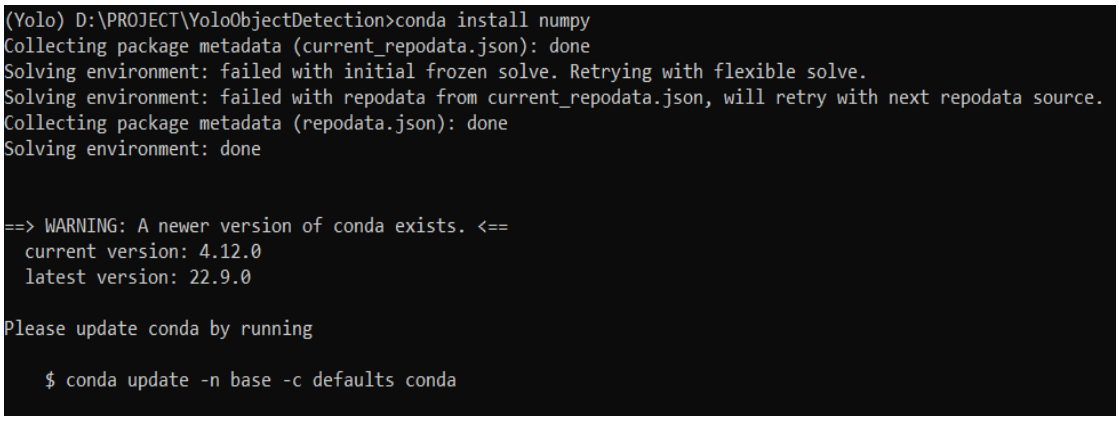
e. Click Yolo environment >> open terminal



f. (Yolo) C:\Users\User>d:
(Yolo) D:\>cd D:\PROJECT\YoloObjectDetection
(Yolo) D:\PROJECT\YoloObjectDetection>



g. Install the necessary packages:
(Yolo) D:\PROJECT\YoloObjectDetection>conda install numpy



```
 blas              pkgs/main/win-64::blas-1.0-mkl
 intel-openmp      pkgs/main/win-64::intel-openmp-2022.1.0-h59b6b97_3788
 mkl               pkgs/main/win-64::mkl-2020.2-256
 mkl-service       pkgs/main/win-64::mkl-service-2.3.0-py36h196d8e1_0
 mkl_fft           pkgs/main/win-64::mkl_fft-1.3.0-py36h46781fe_0
 mkl_random        pkgs/main/win-64::mkl_random-1.1.1-py36h47e9c7a_0
 numpy             pkgs/main/win-64::numpy-1.19.2-py36hadc3359_0
 numpy-base        pkgs/main/win-64::numpy-base-1.19.2-py36ha3acd2a_0
 six               pkgs/main/noarch::six-1.16.0-pyhd3eb1b0_1


Proceed ([y]/n)? y


Downloading and Extracting Packages
mkl_random-1.1.1     | 235 KB    | ############################################################################ | 100%
numpy-1.19.2         | 22 KB     | ############################################################################ | 100%
mkl_fft-1.3.0        | 131 KB    | ############################################################################ | 100%
mkl-service-2.3.0    | 45 KB     | ############################################################################ | 100%
numpy-base-1.19.2    | 3.8 MB    | ############################################################################ | 100%
mkl-2020.2           | 109.3 MB  | ############################################################################ | 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

h. (Yolo) D:\PROJECT\YoloObjectDetection>conda install opencv

```
C:\Windows\system32\cmd.exe - conda  install numpy - conda  install opencv
                                          Total:        1.3 MB

The following NEW packages will be INSTALLED:

  hdf5               pkgs/main/win-64::hdf5-1.8.20-hac2f561_1
  icc_rt             pkgs/main/win-64::icc_rt-2022.1.0-h6049295_2
  jpeg               pkgs/main/win-64::jpeg-9e-h2bbff1b_0
  lerc               pkgs/main/win-64::lerc-3.0-hd77b12b_0
  libdeflate         pkgs/main/win-64::libdeflate-1.8-h2bbff1b_5
  libopencv          pkgs/main/win-64::libopencv-3.4.2-h20b85fd_0
  libpng             pkgs/main/win-64::libpng-1.6.37-h2a8f88b_0
  libtiff            pkgs/main/win-64::libtiff-4.4.0-h8a3f274_0
  lz4-c              pkgs/main/win-64::lz4-c-1.9.3-h2bbff1b_1
  opencv             pkgs/main/win-64::opencv-3.4.2-py36h40b0b35_0
  py-opencv          pkgs/main/win-64::py-opencv-3.4.2-py36hc319ecb_0
  xz                 pkgs/main/win-64::xz-5.2.6-h8cc25b3_0
  zlib               pkgs/main/win-64::zlib-1.2.13-h8cc25b3_0
  zstd               pkgs/main/win-64::zstd-1.5.2-h19a0ad4_0


Proceed ([y]/n)? y


Downloading and Extracting Packages
zlib-1.2.13          | 113 KB    | ##########################################
py-opencv-3.4.2      | 1.2 MB    | ##########################################
opencv-3.4.2         | 10 KB     | ##########################################
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

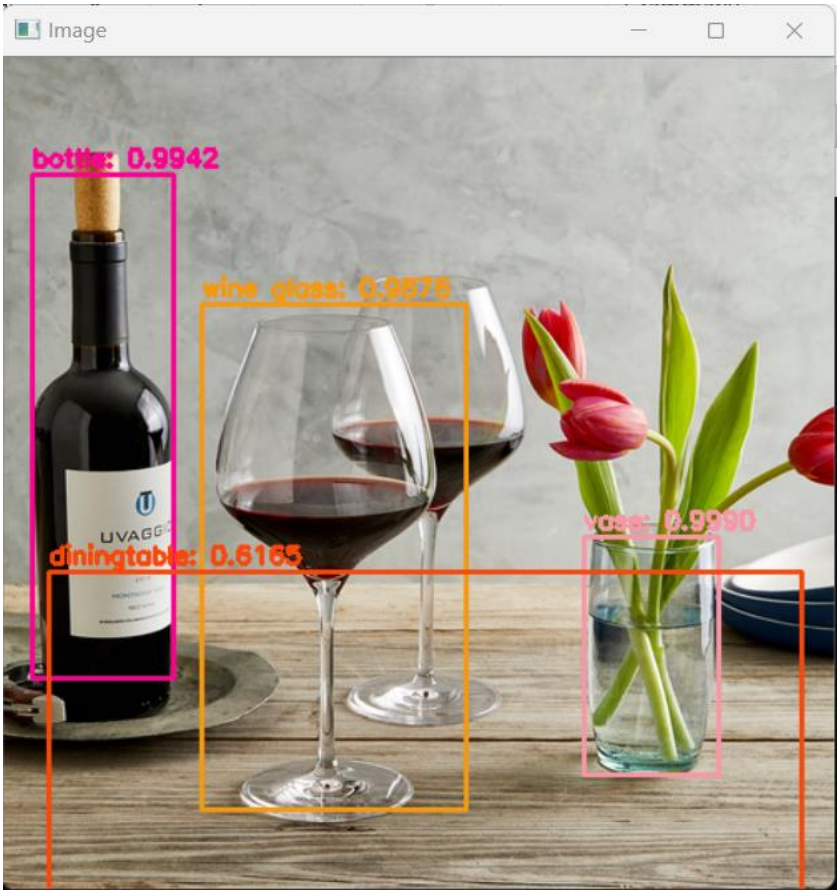
2. **Detect Image**

(Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/a.jpg --yolo yolo-coco

```
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/a.jpg --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] YOLO took 0.527674 seconds

(Yolo) D:\PROJECT\YoloObjectDetection>
```

Output:



3. **Code yolo.py**

```
# USAGE
# python yolo.py --image images/baggage_claim.jpg --yolo yolo-coco

# import the necessary packages
import numpy as np
import argparse
import time
import cv2
import os
```

```
# construct the argument parse and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-i", "--image", required=True,
	help="path to input image")
ap.add_argument("-y", "--yolo", required=True,
	help="base path to YOLO directory")
ap.add_argument("-c", "--confidence", type=float, default=0.5,
	help="minimum probability to filter weak detections")
ap.add_argument("-t", "--threshold", type=float, default=0.3,
	help="threshold when applyong non-maxima suppression")
args = vars(ap.parse_args())

# load the COCO class labels our YOLO model was trained on
labelsPath = os.path.sep.join([args["yolo"], "coco.names"])
LABELS = open(labelsPath).read().strip().split("\n")

# initialize a list of colors to represent each possible class label
np.random.seed(42)
COLORS = np.random.randint(0, 255, size=(len(LABELS), 3),
	dtype="uint8")

# derive the paths to the YOLO weights and model configuration
weightsPath = os.path.sep.join([args["yolo"], "yolov3.weights"])
configPath = os.path.sep.join([args["yolo"], "yolov3.cfg"])

# load our YOLO object detector trained on COCO dataset (80 classes)
print("[INFO] loading YOLO from disk...")
net = cv2.dnn.readNetFromDarknet(configPath, weightsPath)

# load our input image and grab its spatial dimensions
image = cv2.imread(args["image"])
(H, W) = image.shape[:2]

# determine only the *output* layer names that we need from YOLO
ln = net.getLayerNames()
ln = [ln[i[0] - 1] for i in net.getUnconnectedOutLayers()]

# construct a blob from the input image and then perform a forward
# pass of the YOLO object detector, giving us our bounding boxes and
# associated probabilities
blob = cv2.dnn.blobFromImage(image, 1 / 255.0, (416, 416),
	swapRB=True, crop=False)
net.setInput(blob)
start = time.time()
layerOutputs = net.forward(ln)
end = time.time()

# show timing information on YOLO
print("[INFO] YOLO took {:.6f} seconds".format(end - start))

# initialize our lists of detected bounding boxes, confidences, and
# class IDs, respectively
boxes = []
confidences = []
classIDs = []
```

```
# loop over each of the layer outputs
for output in layerOutputs:
      # loop over each of the detections
      for detection in output:
# extract the class ID and confidence (i.e., probability) of
# the current object detection
            scores = detection[5:]
            classID = np.argmax(scores)
            confidence = scores[classID]

# filter out weak predictions by ensuring the detected
# probability is greater than the minimum probability
            if confidence > args["confidence"]:
# scale the bounding box coordinates back relative to the
# size of the image, keeping in mind that YOLO actually
# returns the center (x, y)-coordinates of the bounding
# box followed by the boxes' width and height
                  box = detection[0:4] * np.array([W, H, W, H])
                  (centerX, centerY, width, height) = box.astype("int")
# use the center (x, y)-coordinates to derive the top and
# and left corner of the bounding box
                  x = int(centerX - (width / 2))
                  y = int(centerY - (height / 2))

# update our list of bounding box coordinates, confidences,
# and class IDs
                  boxes.append([x, y, int(width), int(height)])
                  confidences.append(float(confidence))
                  classIDs.append(classID)

# apply non-maxima suppression to suppress weak, overlapping bounding
# boxes
idxs = cv2.dnn.NMSBoxes(boxes, confidences, args["confidence"],
      args["threshold"])

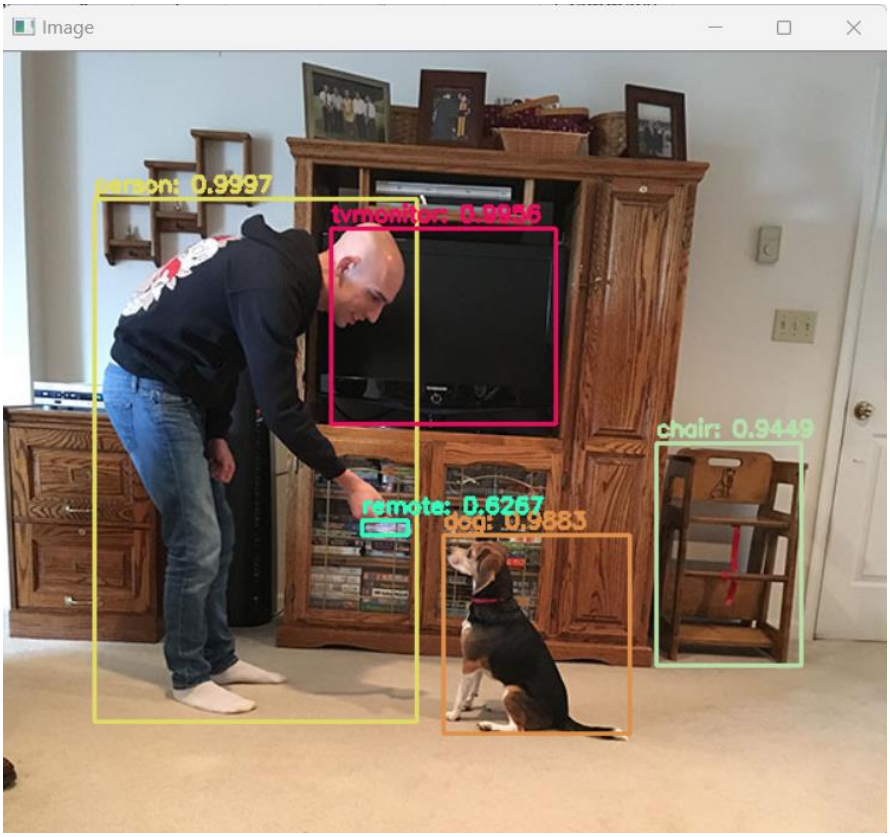
# ensure at least one detection exists
if len(idxs) > 0:
      # loop over the indexes we are keeping
      for i in idxs.flatten():
            # extract the bounding box coordinates
            (x, y) = (boxes[i][0], boxes[i][1])
            (w, h) = (boxes[i][2], boxes[i][3])

            # draw a bounding box rectangle and label on the image
            color = [int(c) for c in COLORS[classIDs[i]]]
            cv2.rectangle(image, (x, y), (x + w, y + h), color, 2)
            text = "{}: {:.4f}".format(LABELS[classIDs[i]],
confidences[i])
            cv2.putText(image, text, (x, y - 5),
cv2.FONT_HERSHEY_SIMPLEX,
                  0.5, color, 2)

# show the output image
cv2.imshow("Image", image)
cv2.waitKey(0)
```

4. Try another image:
   (Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/living_room.jpg --yolo yolo-coco

```
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/living_room.jpg --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] YOLO took 0.647803 seconds
```



   (Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/baggage_claim.jpg --yolo yolo-coco

```
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo.py --image images/baggage_claim.jpg --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] YOLO took 0.533274 seconds
```



5. **Practice 1:**
   Pleased find 10 images based on the list of coco.names and try to detect using yolo!
   - coco.names

coco.names

```
1  person
2  bicycle
3  car
4  motorbike
5  aeroplane
6  bus
7  train
8  truck
9  boat
10 traffic light
11 fire hydrant
12 stop sign
13 parking meter
14 bench
15 bird
16 cat
17 dog
18 horse
19 sheep
20 cow
```

coco.names

```
21 elephant
22 bear
23 zebra
24 giraffe
25 backpack
26 umbrella
27 handbag
28 tie
29 suitcase
30 frisbee
31 skis
32 snowboard
33 sports ball
34 kite
35 baseball bat
36 baseball glove
37 skateboard
38 surfboard
39 tennis racket
40 bottle
```

coco.names

```
41 wine glass
42 cup
43 fork
44 knife
45 spoon
46 bowl
47 banana
48 apple
49 sandwich
50 orange
51 broccoli
52 carrot
53 hot dog
54 pizza
55 donut
56 cake
57 chair
58 sofa
59 pottedplant
60 bed
```

coco.names

```
61 diningtable
62 toilet
63 tvmonitor
64 laptop
65 mouse
66 remote
67 keyboard
68 cell phone
69 microwave
70 oven
71 toaster
72 sink
73 refrigerator
74 book
75 clock
76 vase
77 scissors
78 teddy bear
79 hair drier
80 toothbrush
```

6. **Video Recognition**
   a. **Install library**

(Yolo) D:\PROJECT\YoloObjectDetection>pip install imutils

```
(Yolo) D:\PROJECT\YoloObjectDetection>pip install imutils
Collecting imutils
  Using cached imutils-0.5.4.tar.gz (17 kB)
Building wheels for collected packages: imutils
  Building wheel for imutils (setup.py) ... done
  Created wheel for imutils: filename=imutils-0.5.4-py3-none-any.whl size=25858 sha256=9053e245cd98eb628883e51abcccb45a
436419a6b11a0a7a5ebedefa7e75640f
  Stored in directory: c:\users\user\appdata\local\pip\cache\wheels\f5\0c\3a\61b992f7aa85de40f339e6d4970d91dddb103dd0ad
6c5d58f2
Successfully built imutils
Installing collected packages: imutils
Successfully installed imutils-0.5.4
```

b. (Yolo) D:\PROJECT\YoloObjectDetection>python yolo_video.py --input videos/airport.mp4 --output output/airport_output.avi --yolo yolo-coco
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo_video.py --input videos/airport.mp4 --output output/airport_output.avi --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] 749 total frames in video
[INFO] single frame took 0.5667 seconds
[INFO] estimated total time to finish: 424.4713
[INFO] cleaning up...

```
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo_video.py --input videos/airport.mp4 --output output/airport_output.av
i --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] 749 total frames in video
[INFO] single frame took 0.5667 seconds
[INFO] estimated total time to finish: 424.4713
[INFO] cleaning up...
```

c. Output





7. **yolo_video.py**

```
# USAGE
# python yolo_video.py --input videos/airport.mp4 --output
output/airport_output.avi --yolo yolo-coco

# import the necessary packages
import numpy as np
import argparse
```

```
import imutils
import time
import cv2
import os

# construct the argument parse and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-i", "--input", required=True,
	help="path to input video")
ap.add_argument("-o", "--output", required=True,
	help="path to output video")
ap.add_argument("-y", "--yolo", required=True,
	help="base path to YOLO directory")
ap.add_argument("-c", "--confidence", type=float, default=0.5,
	help="minimum probability to filter weak detections")
ap.add_argument("-t", "--threshold", type=float, default=0.3,
	help="threshold when applyong non-maxima suppression")
args = vars(ap.parse_args())

# load the COCO class labels our YOLO model was trained on
labelsPath = os.path.sep.join([args["yolo"], "coco.names"])
LABELS = open(labelsPath).read().strip().split("\n")

# initialize a list of colors to represent each possible class label
np.random.seed(42)
COLORS = np.random.randint(0, 255, size=(len(LABELS), 3),
	dtype="uint8")

# derive the paths to the YOLO weights and model configuration
weightsPath = os.path.sep.join([args["yolo"], "yolov3.weights"])
configPath = os.path.sep.join([args["yolo"], "yolov3.cfg"])

# load our YOLO object detector trained on COCO dataset (80 classes)
# and determine only the *output* layer names that we need from YOLO
print("[INFO] loading YOLO from disk...")
net = cv2.dnn.readNetFromDarknet(configPath, weightsPath)
ln = net.getLayerNames()
ln = [ln[i[0] - 1] for i in net.getUnconnectedOutLayers()]

# initialize the video stream, pointer to output video file, and
# frame dimensions
vs = cv2.VideoCapture(args["input"])
writer = None
(W, H) = (None, None)

# try to determine the total number of frames in the video file
try:
	prop = cv2.cv.CV_CAP_PROP_FRAME_COUNT if imutils.is_cv2() \
		else cv2.CAP_PROP_FRAME_COUNT
	total = int(vs.get(prop))
	print("[INFO] {} total frames in video".format(total))

# an error occurred while trying to determine the total
# number of frames in the video file
except:
	print("[INFO] could not determine # of frames in video")
```

```
	print("[INFO] no approx. completion time can be provided")
	total = -1

# loop over frames from the video file stream
while True:
	# read the next frame from the file
	(grabbed, frame) = vs.read()

	# if the frame was not grabbed, then we have reached the end
	# of the stream
	if not grabbed:
		break

	# if the frame dimensions are empty, grab them
	if W is None or H is None:
		(H, W) = frame.shape[:2]

	# construct a blob from the input frame and then perform a
forward
	# pass of the YOLO object detector, giving us our bounding boxes
	# and associated probabilities
	blob = cv2.dnn.blobFromImage(frame, 1 / 255.0, (416, 416),
		swapRB=True, crop=False)
	net.setInput(blob)
	start = time.time()
	layerOutputs = net.forward(ln)
	end = time.time()

	# initialize our lists of detected bounding boxes, confidences,
	# and class IDs, respectively
	boxes = []
	confidences = []
	classIDs = []

	# loop over each of the layer outputs
	for output in layerOutputs:
		# loop over each of the detections
		for detection in output:
			# extract the class ID and confidence (i.e.,
probability)
			# of the current object detection
			scores = detection[5:]
			classID = np.argmax(scores)
			confidence = scores[classID]

			# filter out weak predictions by ensuring the
detected
			# probability is greater than the minimum probability
			if confidence > args["confidence"]:
				# scale the bounding box coordinates back
relative to
				# the size of the image, keeping in mind that
YOLO
# actually returns the center (x, y)-coordinates of
# the bounding box followed by the boxes' width and
# height
```

```
                        box = detection[0:4] * np.array([W, H, W, H])
                        (centerX, centerY, width, height) =
box.astype("int")

# use the center (x, y)-coordinates to derive the top
# and and left corner of the bounding box
                        x = int(centerX - (width / 2))
                        y = int(centerY - (height / 2))

# update our list of bounding box coordinates,
# confidences, and class IDs
                        boxes.append([x, y, int(width), int(height)])
                        confidences.append(float(confidence))
                        classIDs.append(classID)

      # apply non-maxima suppression to suppress weak, overlapping
      # bounding boxes
      idxs = cv2.dnn.NMSBoxes(boxes, confidences, args["confidence"],
            args["threshold"])

      # ensure at least one detection exists
      if len(idxs) > 0:
            # loop over the indexes we are keeping
            for i in idxs.flatten():
                  # extract the bounding box coordinates
                  (x, y) = (boxes[i][0], boxes[i][1])
                  (w, h) = (boxes[i][2], boxes[i][3])

                  # draw a bounding box rectangle and label on the
frame
                  color = [int(c) for c in COLORS[classIDs[i]]]
                  cv2.rectangle(frame, (x, y), (x + w, y + h), color,
2)
                  text = "{}: {:.4f}".format(LABELS[classIDs[i]],
                        confidences[i])
                  cv2.putText(frame, text, (x, y - 5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

      # check if the video writer is None
      if writer is None:
            # initialize our video writer
            fourcc = cv2.VideoWriter_fourcc(*"MJPG")
            writer = cv2.VideoWriter(args["output"], fourcc, 30,
                  (frame.shape[1], frame.shape[0]), True)

            # some information on processing single frame
            if total > 0:
                  elap = (end - start)
                  print("[INFO] single frame took {:.4f}
seconds".format(elap))
                  print("[INFO] estimated total time to finish:
{:.4f}".format(
                        elap * total))

      # write the output frame to disk
      writer.write(frame)
```

```
# release the file pointers
print("[INFO] cleaning up...")
writer.release()
vs.release()
```

8. **Try another video:**

(Yolo) D:\PROJECT\YoloObjectDetection>python yolo_video.py --input videos/car_chase_01.mp4 --output output/car_chase_01_output.avi --yolo yolo-coco
(Yolo) D:\PROJECT\YoloObjectDetection>python yolo_video.py --input videos/car_chase_01.mp4 --output output/car_chase_01_output.avi --yolo yolo-coco
[INFO] loading YOLO from disk...
[INFO] 350 total frames in video
[INFO] single frame took 0.5411 seconds
[INFO] estimated total time to finish: 189.3978
[INFO] cleaning up...

Output:





9. **Practice 2:**
   Pleased find 2 short videos based on the list of coco.names and try to detect using yolo!

Reference:

https://pyimagesearch.com/2018/11/12/yolo-object-detection-with-opencv/